



TÉCNICO LISBOA

TECNOLOGIA E PROJECTO DE ANTENAS

MEEC

The folded dipole

Autores:

Francisco Matias (99936)
Miguel Lourenço (100037)

francisco.m.matias@tecnico.ulisboa.pt
miguel.c.lourenco@tecnico.ulisboa.pt

Grupo 7

2023/2024 – 2º Semestre, P4

Conteúdo

1	Q1	2
2	Q2	4
3	Q3	5
4	Q4	7
5	Q5	10
6	Q6	11
7	Anexos	14
7.1	Q1	14
7.2	Q2	15
7.3	Q3	17
7.4	Q4	19
7.5	Q5	20
8	Referências	23

1 Q1

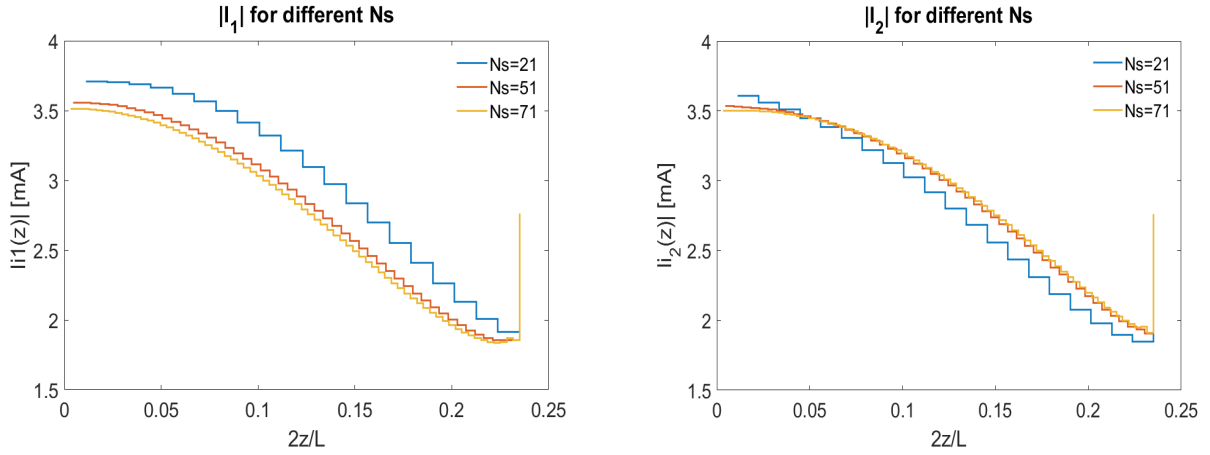


Figura 1: $|I_1|$ and $|I_2|$ para a folded dipole antenna com $L = 0.47\lambda$, para $N_s = 21, 51, 71$

Métodos Numéricos e Convergência

O tamanho do step δ é calculado como $\delta = L/(2N_s)$. Um step menor (maior N_s) resulta em uma integração numérica mais precisa e uma melhor resolução da distribuição de corrente. O uso de valores maiores de N_s , como 51 e 71, leva a resultados mais convergentes, indicando que a solução está mais próxima do comportamento físico real das correntes da antena.

No código fornecido (ver Anexo), o problema é formulado utilizando matrizes de impedância Z_s e Z_c , que representam impedâncias próprias e mútuas, respectivamente. Ao resolver o sistema de equações lineares derivado dessas matrizes, as correntes I_1 e I_2 são obtidas.

Comportamento das Correntes

Os gráficos da Figura 1 mostram uma diminuição gradual no valor absoluto das correntes $|I_1(z)|$ e $|I_2(z)|$ à medida que $2z/L$ aumenta de 0 a 0.25. Este comportamento é esperado, pois a distribuição de corrente em uma antena dipolo normalmente atinge o pico no ponto de alimentação (centro da antena) e diminui em direção às extremidades devido ao padrão de onda estacionária formado pelas ondas refletidas.

À medida que o número de steps N_s aumenta de 21 para 71, as curvas tornam-se mais suaves e contínuas. Isso indica uma maior resolução na solução numérica.

Com N_s menor (por exemplo, 21), os valores de corrente são mais segmentados, mostrando mudanças abruptas. À medida que N_s aumenta, esses segmentos tornam-se mais finos, levando a uma representação mais precisa da distribuição contínua de corrente.

Os padrões de $|I_1(z)|$ e $|I_2(z)|$ são semelhantes, mas não idênticos. Isso se deve ao layout físico da antena dipolo dobrado, onde as correntes nos dois condutores (denotadas como I_1 e I_2) não são iguais devido à dobra e ao acoplamento mútuo entre os condutores.

Concluindo, os gráficos demonstram de forma eficaz como o aumento do número de steps N_s melhora a precisão da solução numérica para a distribuição de corrente em uma antena folded dipole. Esta maior precisão é crítica para o projeto e otimização confiáveis de sistemas de antenas. As curvas mais suaves e graduais para valores mais altos de N_s indicam melhor convergência e modelagem mais realista do comportamento físico da antena.

2 Q2

Análise de Resultados

Sabemos que a impedância de entrada teórica de um folded dipole é 4 vezes a impedância de entrada de um dipolo de meia onda, que é aproximadamente $73 + j42\Omega$. Portanto, a impedância esperada é:

$$Z_{\text{folded dipole}} \approx 4 \times (73 + j42) \approx 292 + j168\Omega$$

Os resultados obtidos para a impedância de entrada (Z_{in}) são:

N_s	Z_{in}	R_{in}	X_{in}
21	258.75 - 70.32j	258.75	70.32
51	279.20 - 32.82j	279.20	32.82
71	283.64 - 23.62j	283.64	23.62

Tabela 1: Impedância de entrada para a folded dipole antenna com $L = 0.47\lambda$, para $N_s = 21, 51, 71$

Parte Real (Resistência)

Observamos que a parte real da impedância de entrada aumenta com o aumento de N_s .

Para $N_s = 71$, a resistência aproxima-se do valor teórico de 292Ω , indicando uma melhoria na precisão da simulação com o aumento da resolução.

Parte Imaginária (Reatância)

A parte imaginária da impedância de entrada diminui com o aumento de N_s .

Para $N_s = 71$, a reatância é de -23.62Ω , aproximando-se da reatância teórica de 168Ω , embora ainda haja uma diferença significativa.

Módulo da Impedância

O módulo da impedância de entrada também aumenta com N_s , aproximando-se dos valores teóricos.

Esta análise confirma que aumentar a resolução do modelo numérico (aumentar N_s) resulta em uma melhor aproximação do comportamento físico real da antena do folded dipole. No entanto, ainda existem diferenças entre os valores teóricos e os valores obtidos, especialmente na parte imaginária da impedância, que podem ser atribuídas a simplificações e limitações do modelo utilizado.

3 Q3

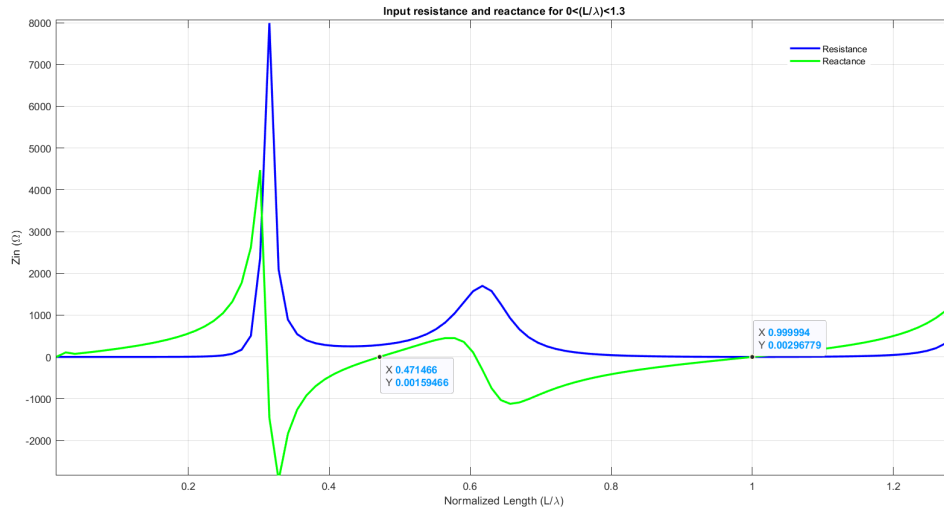


Figura 2: Resistência de entrada e reatância de entrada do dipolo excitado na faixa $0 < L/\lambda < 1.3$, para $N_s = 51$.

Para representar graficamente a resistência e a reatância de entrada de um dipolo alimentado no intervalo $0 < L/\lambda < 1.3$, inicialmente, é necessário calcular a impedância de entrada para diferentes comprimentos do dipolo. Isso pode ser feito utilizando métodos numéricos, como integração numérica, conforme mostrado no código fornecido (ver Anexos).

Ao dar plot no gráfico (Figura 2), podemos observar dois pontos onde a parte imaginária da impedância de entrada atinge zero. Esses pontos correspondem às frequências de ressonância do dipolo, onde a antena está a operar na sua forma mais eficiente.

Os valores de L/λ são:

- $L/\lambda = 0.471466$
- $L/\lambda \approx 1$

Dado que $\lambda = 15$ cm, podemos calcular os valores correspondentes de L para ressonância:

1. Para $L/\lambda = 0.471466$:

$$L = (0.471466) \times 15 \text{ cm} = 7.07199 \text{ cm}$$

2. Para $L/\lambda \approx 1$:

$$L = (1) \times 15 \text{ cm} = 15 \text{ cm}$$

Portanto, os valores de L para ressonância são aproximadamente 7.07199 cm e 15 cm, respectivamente.

Os valores de L identificados como pontos de ressonância representam os comprimentos do dipolo nos quais a antena é mais ressonante e, portanto, mais eficiente em termos de transmissão ou recepção de sinais. Estes valores de L são cruciais para otimizar o desempenho da antena em sua aplicação específica.

É ainda visível que por volta de $L/\lambda = 0.31$ a resistência tende para infinito, o que está de acordo com o esperado teoricamente, como vamos poder confirmar mais à frente neste trabalho (Q6).

4 Q4

Pretende-se representar o diagrama de radiação do dipolo dobrado nos planos xoy e xoz para $L = 0.1\lambda$, $L = 0.47\lambda$ e $L = 1.1\lambda$. Especificamente, iremos projetar um gráfico polar do valor absoluto da altura efetiva da antena, dada pela expressão fornecida.

A altura efetiva $h_e(\theta)$ da antena é uma medida que relaciona a intensidade do campo elétrico na direção θ com a corrente no ponto de alimentação da antena. A expressão para a altura efetiva é dada por:

$$h_e(\theta) = \sin \theta \int_{-L/2}^{L/2} dz' \frac{I_1(z') + I_2(z')}{I_1(0)} e^{jk_0 z' \cos \theta} \quad (4.0.1)$$

que pode ser simplificada para:

$$h_e(\theta) = 2 \sin \theta \int_0^{L/2} dz' \frac{I_1(z') + I_2(z')}{I_1(0)} \cos(k_0 z' \cos \theta) \quad (4.0.2)$$

Além disso, para $L = 0.47\lambda$, é necessário encontrar o HPBW (Largura de Feixe a Meia Potência).

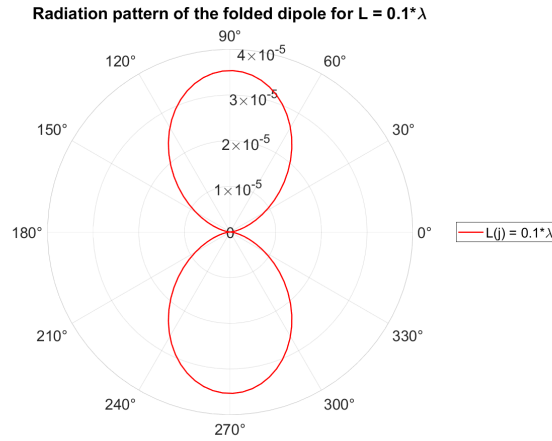


Figura 3: Diagrama de radiação do Dipolo Dobrado nos planos xoy e xoz para $L = 0.1\lambda$

No primeiro caso, o ganho é aproximadamente 3.5×10^{-5} para um $\theta = 90^\circ$.

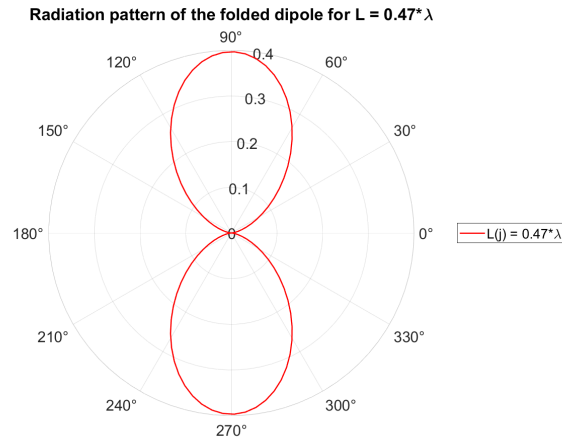


Figura 4: Diagrama de radiação do Dipolo Dobrado nos planos xoy e xoz para $L = 0.47\lambda$

Quando $L = 0.47\lambda$ o ganho é aproximadamente 0,403 para um $\theta = 90^\circ$. Verificou-se ainda um lóbulo principal bem definido e menores lóbulos secundários.

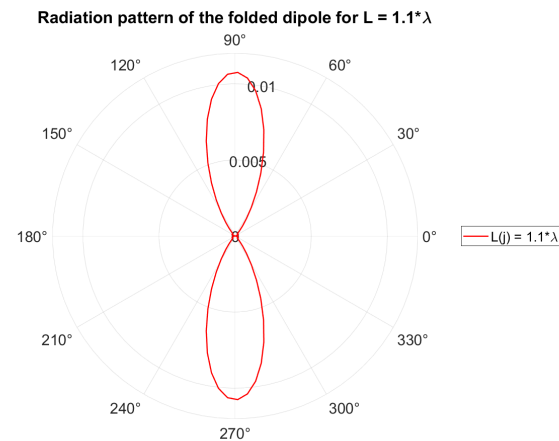


Figura 5: Diagrama de radiação do Dipolo Dobrado nos planos xoy e xoz para $L = 1.1\lambda$

Neste último caso, o ganho é aproximadamente 0.0107 para um $\theta = 90^\circ$.

No plano xoz o diagrama de radiação apresenta semelhanças com o do dipolo de Hertz apresentando o máximo para $\theta = 90^\circ$.

Nos três diagramas apresentados, é notável que o maior ganho ocorre para $L = 0.47\lambda$ com um valor de 0.403, para $\theta = 90^\circ$. Também se observa que para $L = 1.1\lambda$, o diagrama apresenta distorções para ângulos θ baixos.

No cálculo do HPBW (Largura de Feixe a Meia Potência), foram encontrados os pontos mais próximos de $\sqrt{\frac{1}{2}}$ do ganho máximo com $L = 0.47\lambda$. De acordo com o diagrama de radiação, o **HPBW** é igual a **78°** ($128^\circ - 50^\circ$).

Apenas uma última nota, analisando o diagrama de radiação no plano xoy , este mostrou-se isotrópico, com um ganho de 1 para qualquer ângulo, independentemente da relação entre a sua largura e o comprimento de onda.

5 Q5

Nesta questão, pretende-se representar graficamente a diretividade de uma antena em função da razão $\frac{L}{\lambda}$ para o intervalo de 0.1 a 1.1. Para o cálculo da diretividade, iremos utilizar a fórmula fornecida, que envolve a integração da função do padrão de radiação da antena.

A diretividade D de uma antena é uma medida da concentração da radiação emitida numa direção particular em comparação com a radiação emitida uniformemente em todas as direções. A expressão para a diretividade é dada por:

$$D = \frac{2|h_e(\theta = 90^\circ)|^2}{\int_0^\pi |h_e(\theta)|^2 \sin \theta d\theta} \quad (5.0.1)$$

Esta fórmula permite calcular a diretividade utilizando a função do padrão de radiação $h_e(\theta)$, que descreve a variação da intensidade do campo elétrico emitido pela antena em diferentes direções.

De seguida está representado o gráfico da diretividade da antena:

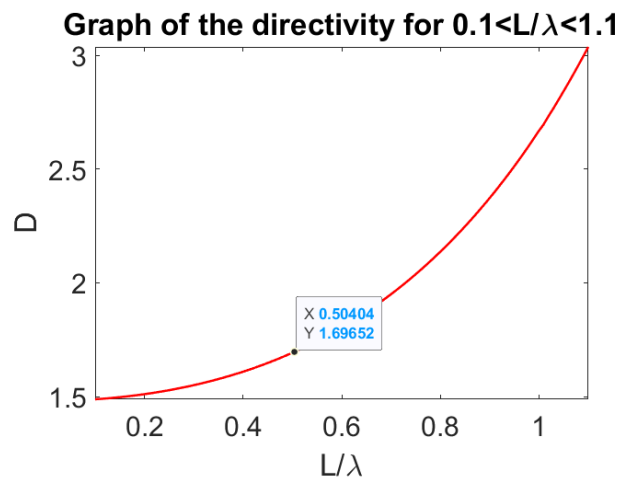


Figura 6: Diretividade da Antena como função de L/λ for $0 < L/\lambda < 1.1$

Podemos confirmar que, conforme esperado, a diretividade aumenta à medida que a relação $\frac{L}{\lambda}$ cresce. Este comportamento aproxima-se do de um dipolo de meio comprimento de onda, como se pode constatar com o valor da diretividade de 1.66, que está muito próximo do valor teórico de 1.64.

6 Q6

Podemos analisar o dipolo dobrado como uma linha de transmissão com uma terminação em curto-circuito. Neste caso, ele é alimentado com metade da tensão, mas com sinais opostos (modo de transmissão). Por outro lado, no modo antena, o dipolo dobrado é alimentado com duas tensões no mesmo sentido. Tal é possível verificar conforme é mostrado na seguinte figura:

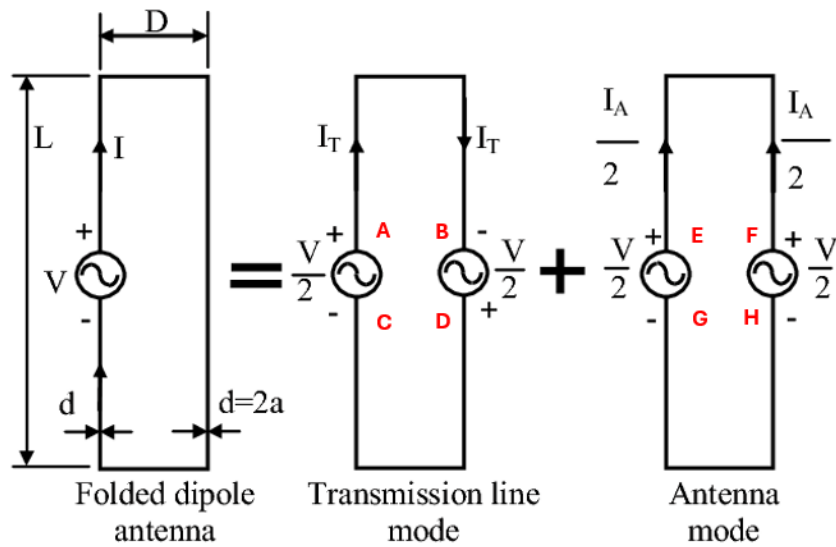


Figura 7: Decomposição do dipolo dobrado no modo de transmissão e no modo antena

Para uma linha de transmissão, podemos calcular a impedância com a seguinte equação:

$$Z_t = \frac{Z_l + jZ_0 \tan(kd)}{Z_0 + jZ_l \tan(kd)} \quad (6.0.1)$$

Em que Z_l é a impedância da terminação da linha que, no caso do modo de transmissão, é $Z_l = 0$. Z_0 representa a impedância característica de duas linhas paralelas. O comprimento da linha é $l = \frac{\lambda}{2}$, logo:

$$Z_t = jZ_0 \tan\left(k\frac{\lambda}{2}\right) \quad (6.0.2)$$

Através da figura 5 (Transmission line mode) detetámos que entre dois pontos da linha de transmissão (A-B) há uma queda de tensão de $\frac{V}{2}$. Aplicando a lei de

Ohm, temos a relação:

$$I_t = \frac{V}{2Z_t} \quad (6.0.3)$$

Considerando Z_t como a impedância da linha de transmissão. Através da Figura 5 (Antenna mode), notamos que a tensão nos pontos E-F e G-H é igual, permitindo-nos aproximar por um dipolo alimentado por $\frac{V}{2}$.

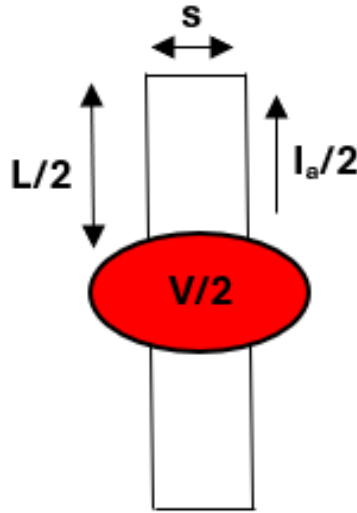


Figura 8: Aproximação do modo antenna por um dipolo com 2 braços.

Quando $s \ll \lambda$, podemos considerar que as duas correntes $\frac{I_a}{2}$ se somam, levando à seguinte expressão pela lei de Ohm:

$$I_a = \frac{V}{2Z_d} \quad (6.0.4)$$

Onde Z_d representa a impedância do dipolo de $\frac{\lambda}{2}$. A corrente de entrada do dipolo é então calculada como:

$$I_{in} = I_t + I_z = \frac{V(2Z_d + Z_t)}{4Z_tZ_d} \quad (6.0.5)$$

Isto resulta na seguinte expressão para a impedância de entrada:

$$Z_{in} = \frac{V}{I_{in}} = \frac{4Z_tZ_d}{2Z_d + Z_t} \quad (6.0.6)$$

Para $L = \frac{\lambda}{2}$ e utilizando a equação (6.0.2), obtemos que $Z_t = jZ_0 \tan(k\frac{\lambda}{2}) = jZ_0$, logo:

$$Z_{in} = \lim_{Z_t \rightarrow jZ_0} \frac{4Z_t Z_d}{2Z_d + Z_t} = \frac{4Z_d Z_0}{2Z_d + jZ_0} \approx 4Z_d \quad (6.0.7)$$

Desta forma, demonstramos que a impedância de entrada de um dipolo dobrado é aproximadamente **4 vezes maior** que a do dipolo de $\frac{\lambda}{2}$. As correntes I_a e I_z podem ser aproximadas pelas equações das correntes dos dipolos no modo antena (Figura 5 - Antenna mode) do dipolo dobrado. Um dipolo de $\frac{\lambda}{2}$ pode ser modelado como uma linha de transmissão com extremidade aberta, resultando na seguinte equação:

$$I(z) = I_m \sin \left(k_0 \left(\frac{\lambda}{2} - |z| \right) \right) \quad (6.0.8)$$

Ao analisar a impedância de entrada para $\frac{\lambda}{4} = 0$, usando as equações (6.0.2) e (6.0.6), obtemos $\mathbf{Z}_t = \mathbf{0}$ e $\mathbf{Z}_{in} = \frac{2\mathbf{Z}_d}{2} = \mathbf{0}$. Para $\frac{\lambda}{8} = 0.25$, podemos utilizar uma abordagem diferente, tratando o dipolo dobrado como uma única linha de transmissão:

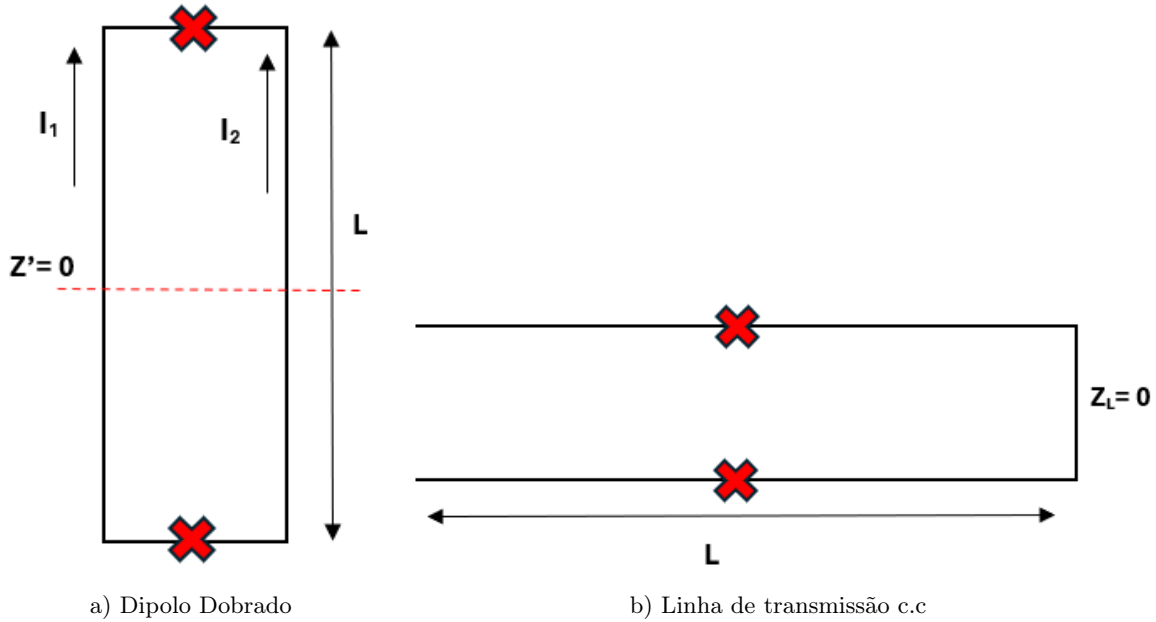


Figura 9: Estimativa de um dipolo dobrado numa linha de transmissão terminada em curto-circuito.

Para linhas de transmissão com comprimentos da forma $L = \frac{\lambda}{4} + n\frac{\lambda}{2}$, onde $n \in \mathbb{N}$, a impedância de entrada pode ser calculada pela seguinte fórmula:

$$Z_{in} = \frac{Z_0^2}{Z_L} \quad (6.0.9)$$

No caso de $\mathbf{Z}_L = \mathbf{0}$, temos $\mathbf{Z}_{in} = \infty$.

7 Anexos

7.1 Q1

```

1  clear;
2  clf;
3  close all;
4
5
6  % Constants
7  u0 = 4*pi*10^-7;           % Vacuum permeability [H/m]
8  e0 = 8.8542*10^-12;        % Vacuum permittivity [F/m]
9
10 % Given data
11 V = 1;                      % Input voltage [V]
12 lambda = 1;                 % Wavelength [m]
13 a = 0.001*lambda;          % Dipole radius [m]
14 d = 10*a;
15 k0 = (2*pi)/lambda;         % Wave number [rad/m]
16 n0 = sqrt(u0/e0);          % Vacuum impedance
17
18 % Data for ex1 and ex2
19 L = 0.47*lambda;            % Dipole length [m]
20 N = [21, 51, 71];          % Number of steps
21
22 Zin = zeros(1, 3); % Input impedance
23
24 for j = 1:length(N)
25     Ns = N(j);
26     delta = L / (2 * Ns); % Step size
27     zm = zeros(1, Ns); % Segment midpoints
28
29     % Matrices
30     Zs = zeros(Ns, Ns); % Self impedance
31     Zc = zeros(Ns, Ns); % Mutual impedance
32     wc = zeros(Ns, 1);
33     ws = zeros(Ns, 1);
34     column_zeros = zeros(Ns, 1);
35     row_zeros = zeros(1, Ns);
36     row_u = zeros(1, Ns);
37
38     % Calculate segment midpoints
39     for n = 1:Ns
40         zm(n) = (n - 0.5) * delta;
41     end
42
43     for n = 1:Ns
44         % Kernel functions
45         Kc = @(z) ((1 / (4 * pi)) * (exp(-1i * k0 * sqrt(a^2 + (zm(n) - z)^2))) / sqrt(a^2 + (zm(n) - z)^2)) ...
46             + ((1 / (4 * pi)) * (exp(-1i * k0 * sqrt(a^2 + (zm(n) + z)^2))) / sqrt(a^2 + (zm(n) + z)^2)));
47
48         Kself = @(z) ((1 / (4 * pi)) * (exp(-1i * k0 * sqrt(d^2 + (zm(n) - z)^2))) / sqrt(d^2 + (zm(n) - z)^2)) ...
49             + ((1 / (4 * pi)) * (exp(-1i * k0 * sqrt(d^2 + (zm(n) + z)^2))) / sqrt(d^2 + (zm(n) + z)^2)));
50
51         for m = 1:Ns
52             Zs(n, m) = integral(Kself, zm(m) - (delta / 2), zm(m) + (delta / 2), 'ArrayValued', true);
53             Zc(n, m) = integral(Kc, zm(m) - (delta / 2), zm(m) + (delta / 2), 'ArrayValued', true);
54             wc(m, 1) = -cos(k0 * zm(m));
55             ws(m, 1) = sin(k0 * abs(zm(m)));
56
57             if m == Ns
58                 row_u(1, m) = 1;

```

```

59         end
60     end
61 end
62
63 % Constructing matrix 1
64 matrix_1 = [Zs Zc wc column_zeros; Zc Zs column_zeros wc; row_u row_u 0 0; row_zeros
        row_zeros sin(k0 * L / 2) -sin(k0 * L / 2)];
65
66 % Constructing matrix 2
67 pre_matrix = [ws; column_zeros; 0; cos(k0 * L / 2)];
68 matrix_2 = (-1i * V / (2 * n0)) * pre_matrix;
69
70 % Solving system of equations
71 final_results = linsolve(matrix_1, matrix_2);
72
73 % Extracting constants and currents
74 constant1 = final_results(Ns - 1, 1);
75 constant2 = final_results(Ns, 1);
76 current1 = final_results(1:Ns);
77 current2 = final_results(Ns + 1:2 * Ns);
78
79 % Calculating input impedance
80 Zin(j) = V / final_results(1, 1);
81
82 % Generating values for plot
83 values = delta * (1:Ns);
84
85 % Plotting currents
86 figure(1);
87 title('Absolute values of current I_1 for different Ns');
88 set(gca, 'FontSize', 18)
89 stairs(values, abs(current1) * 10^-3, 'LineWidth', 1.5);
90 hold on;
91
92 figure(2);
93 title('Absolute values of current I_2 for different Ns');
94 set(gca, 'FontSize', 18)
95 stairs(values, abs(current2) * 10^-3, 'LineWidth', 1.5);
96 hold on;
97 end
98
99 % Labels and legends for the plots
100 figure(1);
101 legend('Ns=21', 'Ns=51', 'Ns=71');
102 xlabel('2z/L');
103 ylabel('|I_1(z)| [mA]');
104
105 figure(2);
106 legend('Ns=21', 'Ns=51', 'Ns=71');
107 xlabel('2z/L');
108 ylabel('|I_2(z)| [mA]');

```

7.2 Q2

```

1
2 clear;
3 clf;
4 close all;
5
6 %constants:
7 u0 = 4*pi*10^-7;           %vacuum permeability [H/m]
8 e0 = 8.8542*10^-12;        %vacuum permittivity [F/m]
9
10 %dados do enunciado:
11 V = 1;                     %input voltage [V]

```

```

12 lambda = 1; %wavelength [m]
13 a = 0.001*lambda; %dipole radius [m]
14 d = 10*a;
15 k0 = (2*pi)/lambda; %wave number [rad/m]
16 n0 = sqrt(u0/e0); %vacuum impedance
17
18 %dados do ex1 e ex2:
19 L = 0.47*lambda; %dipole length [m]
20 N = [21,51,71]; %number of steps
21
22 Zin = zeros(1,3); %question 2
23
24 for j = 1:length(N) %passar por todos os steps que nos d o
25     Ns = N(j);
26     delta = L/(2*Ns); %perto da formula (7)
27     zm = zeros(1,Ns); %formula (7)
28
29     %matrizes necessarias:
30     Zs = zeros(Ns,Ns); %formula (12a)
31     Zc = zeros(Ns,Ns); %formula (12b)
32     wc = zeros(Ns,1);
33     ws = zeros(Ns,1);
34     column_zeros = zeros(Ns,1);
35     row_zeros = zeros(1,Ns);
36     row_u = zeros(1,Ns);
37
38     %formula (7):
39     for n = 1:Ns
40         zm(n) = (n-0.5)*delta;
41     end
42
43     for n = 1:Ns %formula (6) - Kc e Kself:
44         Kc = @(z)((1/(4*pi))*...
45             (exp((-1i)*k0*sqrt((a^2)+((zm(n)-z)^2))))... % calcular part 1...
46             *(1/sqrt((a^2)+ ((zm(n)-z)^2))))...
47             ...
48             +(((1/(4*pi))*...
49             (exp((-1i)*k0*sqrt((a^2)+((zm(n)+z)^2))))... % calcular part 2...
50             *(1/sqrt((a^2)+ ((zm(n)+z)^2)))));
51
52         Kself = @(z)((1/(4*pi))*...
53             (exp((-1i)*k0*sqrt((d^2)+((zm(n)-z)^2))))... % calcular part 1...
54             *(1/sqrt((d^2)+ ((zm(n)-z)^2))))...
55             ...
56             +(((1/(4*pi))*...
57             (exp((-1i)*k0*sqrt((d^2)+((zm(n)+z)^2))))... % calcular part 2...
58             *(1/sqrt((d^2)+ ((zm(n)+z)^2)))));
59
60         for m = 1:Ns %faz as matrizes do Zc, Zs, wc e ws
61             Zs(n,m) = integral(Kself,zm(m)-(delta/2),zm(m)+(delta/2),'arrayValued',true); %
62             %preencher colunas com os integrais de K
63             Zc(n,m) = integral(Kc,zm(m)-(delta/2),zm(m)+(delta/2),'arrayValued',true); %
64             %preencher colunas com os integrais de K
65             wc(m,1) = -cos(k0*zm(m));
66             ws(m,1) = sin(k0*abs(zm(m)));
67
68             if m == Ns %faz a matriz u
69                 row_u(1,m) = 1;
70             end
71         end
72     end
73
74     %concatenar as matrizes para a matriz 1 + formula (14):
75     matrix_1 = [Zs Zc wc column_zeros; Zc Zs column_zeros wc; row_u row_u 0 0; row_zeros
76                 row_zeros sin(k0*L/2) -sin(k0*L/2)];
77
78     %concatenar as matrizes para a matriz 2 + formula (14):
79     pre_matrix = [ws;column_zeros;0;cos(k0*L/2)];
80     matrix_2 = (-1i*V/(2*n0))*pre_matrix;

```

```

78
79 %sistemas de equacoes:
80 final_results = linsolve(matrix_1,matrix_2);
81
82 %guardar a constant1 e constant2:
83 constant1 = final_results(Ns-1,1);
84 constant2 = final_results(Ns,1);
85
86 %guardar a current1 e current2:
87 current1 = final_results(1:Ns);
88 current2 = final_results(Ns+1:2*Ns);
89
90 Zin(j) = V/final_results(1,1) %question 2
91
92 %grafico:
93 values = zeros(1,Ns);
94 for n = 1:Ns
95     values(n) = delta*n;
96 end
97 figure(1);
98 title('Absolute values of the currents 1 and 2 of a folded dipole antenna for Ns: 21, 51
99     and 71');
100 set(gca,'FontSize',18)
101 stairs(values,abs(current1)*10^-3,'LineWidth', 1.5);
102 hold on;
103 stairs(values,abs(current2)*10^-3,'LineWidth', 1.5);
104 legend('Ns=21: |I1|', 'Ns=21: |I2|', 'Ns=51: |I1|', 'Ns=51: |I2|', 'Ns=71: |I1|', 'Ns=71: |I2|');
105 xlabel('2z/L');
106 ylabel('Ii(z) [mA]');
107 end

```

7.3 Q3

```

1
2 clc;
3 clear;
4
5 wavelength = (3 * 10^-8) / (1 * 10^-9); % comprimento de onda [m]
6 voltage = 1; % Tens o em Volts
7 num_steps = 51; % n mero de passos
8 length_values = linspace(0, 1.3 * wavelength, 100); % Intervalo de L de 0 a 1.3*wavelength
9 vacuum_impedance = 120 * pi; % imped ncia do v cuo
10 wave_number = (2 * pi) / wavelength; % n mero de onda [rad/m]
11 radius = 0.001 * wavelength; % raio do dipolo [m]
12 distance = 10 * radius; % Dist ncia entre os dois bra os
13
14 % Armazenamento para resist ncia e reat ncia
15 resistance_in = zeros(length(length_values), 1);
16 reactance_in = zeros(length(length_values), 1);
17
18 for idx = 1:length(length_values)
19     length_L = length_values(idx);
20     delta_L = length_L / (2 * num_steps);
21
22     % Calcular as matrizes necess rias ao sistema
23     [matrix_ZnS, matrix_ZnC, vector_wc, vector_ws] = calculate_matrices(num_steps, delta_L,
24         wave_number, radius, distance);
25
26     unit_vector = zeros(1, num_steps);
27     unit_vector(num_steps) = 1;
28
29     % Resolu o do sistema de matrizes
30     matrix_A = [matrix_ZnS, matrix_ZnC, vector_wc, zeros(num_steps, 1);
31         matrix_ZnC, matrix_ZnS, zeros(num_steps, 1), vector_wc];

```

```

31         unit_vector, unit_vector, 0, 0;
32         zeros(1, num_steps), zeros(1, num_steps), sin(wave_number * length_L / 2), -
           sin(wave_number * length_L / 2)];
33     vector_C = [vector_ws; zeros(num_steps, 1); 0; cos(wave_number * length_L / 2)];
34     vector_C = -1i * voltage / (2 * vacuum_impedance) * vector_C;
35     results = matrix_A \ vector_C;
36
37     % Guardar os resultados pretendidos
38     current_I1 = results(1:num_steps);
39     current_I2 = results(num_steps + 1:2 * num_steps);
40     coef_C1 = results(2 * num_steps + 1);
41     coef_C2 = results(2 * num_steps + 2);
42
43     impedance_Zin = voltage / current_I1(1); % C lculo da imped ncia
44
45     % Guardar a resist ncia e reat ncia calculados
46     resistance_in(idx) = real(impedance_Zin);
47     reactance_in(idx) = imag(impedance_Zin);
48 end
49
50 % Plot do gr fico resistance_in e reactance_in em fun o de L/wavelength
51 figure;
52 plot(length_values / wavelength, resistance_in, 'b-', 'LineWidth', 2, 'DisplayName', '
    Resistance');
53 hold on;
54 plot(length_values / wavelength, reactance_in, 'g', 'LineWidth', 2, 'DisplayName', 'Reactance'
    );
55 xlabel('Normalized Length (L/\lambda)');
56 ylabel('Zin(\Omega)');
57 legend show;
58 title('0 < (L/\lambda) < 1.3');
59 grid on;
60
61 % Fun o para calcular as matrizes necess rias
62 function [matrix_ZnS, matrix_ZnC, vector_wc, vector_ws] = calculate_matrices(num_steps,
    delta_L, wave_number, radius, distance)
63     matrix_ZnS = zeros(num_steps, num_steps);
64     matrix_ZnC = zeros(num_steps, num_steps);
65     vector_wc = zeros(num_steps, 1);
66     vector_ws = zeros(num_steps, 1);
67
68     for n = 1:num_steps
69         zn = (n - 0.5) * delta_L;
70         for m = 1:num_steps
71             zm = (m - 0.5) * delta_L;
72             Kself = @(z_linha) exp(-1i * wave_number * sqrt(radius^2 + (zm - z_linha).^2)) /
                (4 * pi * sqrt(radius^2 + (zm - z_linha).^2)) + ...
73             exp(-1i * wave_number * sqrt(radius^2 + (zm + z_linha).^2)) / (4 * pi * sqrt(
                radius^2 + (zm + z_linha).^2));
74             matrix_ZnS(n, m) = integral(@(z_linha) Kself(z_linha), zn - delta_L / 2, zn +
                delta_L / 2, 'ArrayValued', true);
75
76             Kc = @(z_linha) exp(-1i * wave_number * sqrt(distance^2 + (zm - z_linha).^2)) / (4
                * pi * sqrt(distance^2 + (zm - z_linha).^2)) + ...
77             exp(-1i * wave_number * sqrt(distance^2 + (zm + z_linha).^2)) / (4 * pi * sqrt
                (distance^2 + (zm + z_linha).^2));
78             matrix_ZnC(n, m) = integral(@(z_linha) Kc(z_linha), zn - delta_L / 2, zn + delta_L
                / 2, 'ArrayValued', true);
79         end
80     end
81     zm = (n - 0.5) * delta_L;
82     vector_wc(n) = -cos(wave_number * zm);
83     vector_ws(n) = sin(wave_number * abs(zm));
84 end
85 end

```

7.4 Q4

```

1 clear;
2 clf;
3 close all;
4
5 % Dados de entrada:
6 vltagem_entrada = 1; % Tens o de entrada [V]
7 comprimento_onda = 1; % Comprimento de onda [m]
8 raio_dipolo = 0.001 * comprimento_onda; % Raio do dipolo [m]
9 distancia_dipolos = 10 * raio_dipolo;
10 numero_onda = (2 * pi) / comprimento_onda; % N mero de onda [rad/m]
11 impedancia_vacu = 120 * pi; % Impedancia do v cuo
12
13 % Valores do comprimento do dipolo para a quest o 4
14 comprimento_dipolo_valores = [0.1 * comprimento_onda, 0.47 * comprimento_onda, 1.1 *
15     comprimento_onda];
16 num_segmentos = 51;
17
18 for j = 1:length(comprimento_dipolo_valores)
19     comprimento_dipolo = comprimento_dipolo_valores(j);
20     num_segmentos_atual = num_segmentos;
21     delta = comprimento_dipolo / (2 * num_segmentos_atual);
22     zm = zeros(1, num_segmentos_atual);
23
24     for n = 1:num_segmentos_atual
25         zm(n) = (n - 0.5) * delta;
26     end
27
28     [matriz_ZnS, matriz_ZnC, vetor_wc, vetor_ws, vetor_u] = calcular_matrizes(
29         num_segmentos_atual, zm, delta, numero_onda, raio_dipolo, distancia_dipolos);
30
31     % Constru o das matrizes
32     matriz_A = [matriz_ZnS matriz_ZnC vetor_wc zeros(num_segmentos_atual, 1); matriz_ZnC
33         matriz_ZnS zeros(num_segmentos_atual, 1) vetor_wc; vetor_u vetor_u 0 0; zeros(1,
34         num_segmentos_atual) zeros(1, num_segmentos_atual) sin(numero_onda *
35         comprimento_dipolo / 2) -sin(numero_onda * comprimento_dipolo / 2)];
36     matriz_B = [vetor_ws; zeros(num_segmentos_atual, 1); 0; cos(numero_onda *
37         comprimento_dipolo / 2)];
38     matriz_C = (-1i * vltagem_entrada / (2 * impedancia_vacu)) * matriz_B;
39
40     % Resolu o do sistema de equa es
41     resultados = matriz_A \ matriz_C;
42
43     % Guardar os resultados
44     coeficiente_C1 = resultados(end-1,1);
45     coeficiente_C2 = resultados(end,1);
46     corrente_I1 = resultados(1:num_segmentos_atual);
47     corrente_I2 = resultados(num_segmentos_atual+1:2*num_segmentos_atual);
48
49     % Calcular e plotar o padr o de radia o
50     valores_interpolados = delta * (1:num_segmentos_atual);
51     angulo_teta = 0:0.01:2 * pi;
52     padrao_radiacao = calcular_radiacao(comprimento_dipolo, num_segmentos_atual,
53         valores_interpolados, corrente_I1, corrente_I2, numero_onda, angulo_teta);
54
55     % Plot polar
56     figure;
57     polarplot(angulo_teta, padrao_radiacao, 'r');
58     title(['Radiation Pattern of the folded dipole for L = ', num2str(comprimento_dipolo /
59         comprimento_onda), '\lambda']);
60     legend(['L(', num2str(j), ') = ', num2str(comprimento_dipolo / comprimento_onda), '*\
61         lambda'], 'Location', 'northeast', 'TextColor', 'black');
62 end
63
64 % Fun o para calcular as matrizes
65 function [matriz_ZnS, matriz_ZnC, vetor_wc, vetor_ws, vetor_u] = calcular_matrizes(
66     num_segmentos_atual, zm, delta, numero_onda, raio_dipolo, distancia_dipolos)

```

```

58   matriz_ZnS = zeros(num_segmentos_atual, num_segmentos_atual);
59   matriz_ZnC = zeros(num_segmentos_atual, num_segmentos_atual);
60   vetor_wc = zeros(num_segmentos_atual, 1);
61   vetor_ws = zeros(num_segmentos_atual, 1);
62   vetor_u = zeros(1, num_segmentos_atual);
63   vetor_u(1, num_segmentos_atual) = 1;
64
65   for n = 1:num_segmentos_atual
66       funcao_Kc = @(z) (1 / (4 * pi)) * (exp(-1i * numero_onda * sqrt(raio_dipolo^2 + (zm(n)
67           - z)^2)) / sqrt(raio_dipolo^2 + (zm(n) - z)^2) + ...
68           exp(-1i * numero_onda * sqrt(raio_dipolo^2 + (zm(n) + z)^2)) / sqrt(raio_dipolo^2
69           + (zm(n) + z)^2));
70
71       funcao_Kself = @(z) (1 / (4 * pi)) * (exp(-1i * numero_onda * sqrt(distancia_dipolos^2
72           + (zm(n) - z)^2)) / sqrt(distancia_dipolos^2 + (zm(n) - z)^2) + ...
73           exp(-1i * numero_onda * sqrt(distancia_dipolos^2 + (zm(n) + z)^2)) / sqrt(
74           distancia_dipolos^2 + (zm(n) + z)^2));
75
76       for m = 1:num_segmentos_atual
77           matriz_ZnS(n, m) = integral(funcao_Kself, zm(m) - (delta / 2), zm(m) + (delta / 2),
78               'ArrayValued', true);
79           matriz_ZnC(n, m) = integral(funcao_Kc, zm(m) - (delta / 2), zm(m) + (delta / 2), '
80               ArrayValued', true);
81           vetor_wc(m, 1) = -cos(numero_onda * zm(m));
82           vetor_ws(m, 1) = sin(numero_onda * abs(zm(m)));
83       end
84   end
85 end
86
87 % Função para calcular o padrão de radiação
88 function padrao_radiacao = calcular_radiacao(comprimento_dipolo, ~, valores_interpolados,
89     corrente_I1, corrente_I2, numero_onda, angulo_teta)
90     corrente_interpolada1 = pchip(valores_interpolados, corrente_I1);
91     corrente_interpolada2 = pchip(valores_interpolados, corrente_I2);
92     funcao_corrente = @(z) ((ppval(corrente_interpolada1, z) + ppval(corrente_interpolada2, z)
93         ) / corrente_I1(1)) * cos(numero_onda * z * cos(angulo_teta));
94     padrao_radiacao = 2 * sin(angulo_teta) .* integral(@(z) funcao_corrente(z), 0,
95         comprimento_dipolo / 2, 'ArrayValued', true);
96     padrao_radiacao = abs(padrao_radiacao).^2; % Valor absoluto de h e ao quadrado
97 end

```

7.5 Q5

```

1   clear;
2   clf;
3   close all;
4
5   %constants:
6   u0 = 4*pi*10^-7;           %vacuum permeability [H/m]
7   e0 = 8.8542*10^-12;        %vacuum permittivity [F/m]
8
9   %dados do enunciado:
10  V = 1;                      %input voltage [V]
11  lambda = 1;                  %wavelength [m]
12  a = 0.001*lambda;           %dipole radius [m]
13  d = 10*a;
14  k0 = (2*pi)/lambda;          %wave number [rad/m]
15  n0 = sqrt(u0/e0);           %vacuum impedance
16
17  %dados do ex5:
18  N = 51;                      %number of steps
19  N_pontos = 100;
20  L_lambda = linspace(0.1,1.1,N_pontos); %linear space of L
21  L = L_lambda*lambda;         %dipole length [m]
22  directivity = zeros(1,N_pontos);

```

```

23
24 for j = 1 : N_pontos
25     Ns = N;
26     delta = L(j)/(2*Ns); %perto da formula (7)
27     zm = zeros(1,Ns); %formula (7)
28
29     %matrizes necessarias:
30     Zs = zeros(Ns,Ns); %formula (12a)
31     Zc = zeros(Ns,Ns); %formula (12b)
32     wc = zeros(Ns,1);
33     ws = zeros(Ns,1);
34     column_zeros = zeros(Ns,1);
35     row_zeros = zeros(1,Ns);
36     row_u = zeros(1,Ns);
37
38     for n = 1:Ns %formula (7)
39         zm(n) = (n-0.5)*delta;
40     end
41
42     for n = 1:Ns %formula (6) - Kc e Kself
43         Kc=@(z)((1/(4*pi))*...
44             (exp((-1i)*k0*sqrt((a^2)+((zm(n)-z)^2)))))... % calcular part 1...
45             *(1/sqrt((a^2)+ ((zm(n)-z)^2)))))...
46             ...
47             +((1/(4*pi))*...
48             (exp((-1i)*k0*sqrt((a^2)+((zm(n)+z)^2)))))... % calcular part 2...
49             *(1/sqrt((a^2)+ ((zm(n)+z)^2)))));
50
51         Kself=@(z)((1/(4*pi))*...
52             (exp((-1i)*k0*sqrt((d^2)+((zm(n)-z)^2)))))... % calcular part 1...
53             *(1/sqrt((d^2)+ ((zm(n)-z)^2)))))...
54             ...
55             +((1/(4*pi))*...
56             (exp((-1i)*k0*sqrt((d^2)+((zm(n)+z)^2)))))... % calcular part 2...
57             *(1/sqrt((d^2)+ ((zm(n)+z)^2)))));
58
59         for m=1:Ns %faz as matrizes do Zc, Zs, wc e ws
60             Zs(n,m)=integral(Kself,zm(m)-(delta/2),zm(m)+(delta/2),'arrayValued',true); %
61             %preencher colunas com os integrais de K
62             Zc(n,m)=integral(Kc,zm(m)-(delta/2),zm(m)+(delta/2),'arrayValued',true); %
63             %preencher colunas com os integrais de K
64             wc(m,1)=-cos(k0*zm(m));
65             ws(m,1)=sin(k0*abs(zm(m)));
66
67             if m==Ns %faz a matriz u
68                 row_u(1,m) = 1;
69             end
70         end
71     end
72
73     %concatenar as matrizes para a matriz 1 + formula (14)
74     matrix_1 = [Zs Zc wc column_zeros; Zc Zs column_zeros wc; row_u row_u 0 0; row_zeros
75                 row_zeros sin(k0*L(j)/2) -sin(k0*L(j)/2)];
76
77     %concatenar as matrizes para a matriz 2 + formula (14)
78     pre_matrix = [ws;column_zeros;0;cos(k0*L(j)/2)];
79     matrix_2 = (-1i*V/(2*n0))*pre_matrix;
80
81     %sistemas de equacoes:
82     final_results = linsolve(matrix_1,matrix_2);
83
84     %guardar a constant1 e constant2:
85     constant1 = final_results(Ns-1,1);
86     constant2 = final_results(Ns,1);
87
88     %guardar a current1 e current2:
89     current1 = final_results(1:Ns);
90     current2 = final_results(Ns+1:2*Ns);
91

```

```

89 %calcular a diretividade:
90 teta = linspace(0,2*pi,N_pontos);
91 he = zeros(size(teta));
92 msin = zeros(size(teta));
93 values = zeros(1,Ns);
94 for n = 1:Ns
95     values(n) = delta*n;
96 end
97 poly_current1 = pchip(values,current1);
98 poly_current2 = pchip(values,current2);
99 for n = 1:length(teta)
100     teta_rad = teta(n);
101     f = @(z) ((ppval(poly_current1,z)+ppval(poly_current2,z))/current1(1))*cos(k0*z*cos(
102         teta_rad));
103     he(n) = 2*sin(teta_rad)*integral(@(z)f(z),0,L(j)/2,"ArrayValued",true);
104     he(n) = abs(he(n)*he(n));
105     msin(n) = sin(teta_rad);
106 end
107
108 numerator = 2*abs(he(25))^2;
109 denominator = 0;
110 for n = 1:50
111     denominator = denominator+abs(he(n))^2*msin(n)*(1/N_pontos);
112 end
113 directivity(j) = numerator/denominator;
114 directivity(j) = directivity(j)/8;
115 end
116
117 %grafico:
118 figure;
119 plot(L, directivity, 'red', 'LineWidth', 1.5);
120 title('Graph of the directivity for 0.1<L/\lambda<1.1');
121 set(gca,'FontSize',18)
122 hold on;
123 xlabel('L/\lambda ');
124 ylabel('D');
125 xlim([0.1 1.1]);

```

8 Referências

- [1] Microsoft Word - Antennas_Project_2024_v2.doc
- [2] TPA_All_Lecture_Notes_31May24.pdf